Python OOP: Parameters, Attributes & self

A focused set of notes for understanding exactly what self, parameters, and attributes mean

Main goal: Stop confusing the parameter name, the object attribute, and self.

1. The Three Things You Must Not Confuse

Consider this method:

```
def __init__(self, name, balance):
    self.name = name
    self.balance = balance
```

Term	Example	What it means
self	self	Reference to the current object
Parameter	name, balance	Local names that receive values when the method is called
Attribute	self.name, self.balance	Data stored on the current object

Most important: self is not an attribute, and name is not the same thing as self.name.

2. Start With a Simple Function

Before OOP, first understand what a normal function parameter is.

```
def greet(name):
    print("Hello", name)

greet("Shrishti")
```

Here, name is a **parameter**. When we call greet("Shrishti"), the value is passed to the parameter.

```
def greet(name):
    print(name)

# During this call:
greet("Shrishti")

# conceptually:
# name → "Shrishti"
```

The parameter is a local name used inside the function to refer to the received value.

3. A Method Is a Function Inside a Class

An instance method needs a reference to the object it is operating on.

```
class BankAccount:

    def show_name(self):
        print(self.name)
```

The first parameter is conventionally written as self. It allows the method to access attributes belonging to the particular object that called it.

4. Can self Be Called Something Else?

Yes. self is a convention, not a reserved Python keyword.

```
class BankAccount:

    def show_name(current_object):
        print(current_object.name)

account1 = BankAccount()
account1.show_name()
```

This can work when called normally. Python does not require the literal word self for this parameter.

However, **using self is strongly recommended** because it is the standard convention and makes the code immediately understandable.

> **Best practice:** Use self, even though another valid parameter name could technically be used.

5. Why Use self?

A method needs to know **which object** it is working with.

```
account1
name = "Shrishti"
balance = 5000

account2
name = "Rahul"
balance = 10000

account1.show_name()
# self → account1

account2.show_name()
# self → account2
```

> **Easy meaning:** self means *this particular object*.

6. What Is a Parameter?

Definition: A parameter is a variable written in a function or method definition that receives a value when the function or method is called.

```
def deposit(self, amount):
    self.balance += amount
```

Parameter	Purpose
self	Refers to the current object
amount	Receives the amount supplied to the method

For account1.deposit(2000), conceptually:

```
self   → account1
amount → 2000
```

7. What Is an Attribute?

Definition: An attribute is data associated with an object or class. An instance attribute represents data stored as part of a particular object.

```
class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
```

Expression	Meaning
self.name	The name attribute belonging to the current object
self.balance	The balance attribute belonging to the current object

For account1, self.name refers to account1.name. For account2, it refers to account2.name.

8. The Most Important Difference: name vs self.name

```
self.name = name
```

Part	Type	Meaning
name	Parameter	The value received by the method
self	Current-object reference	Which object should store/use the data
self.name	Attribute	The name data stored on that object

Read the line as:

```
self.name = name

# Store the received 'name'
# inside the current object's 'name' attribute.
```

9. Step-by-Step: What Happens During Object Creation?

```
class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

account1 = BankAccount("Shrishti", 5000)
```

Step	What happens
1	Python creates a new BankAccount object.
2	Python calls __init__() to initialize that object.
3	self refers to the newly created object (account1).
4	name receives "Shrishti".
5	balance receives 5000.
6	self.name = name stores the value in account1's name attribute.
7	self.balance = balance stores the value in account1's balance attribute.

10. Visualizing self, Parameters and Attributes

```
account1 = BankAccount("Shrishti", 5000)

              __init__(self, name, balance)
                        |
                +-------+---------+
                |       |         |
            self       name     balance
              |         |         |
              v         v         v
          account1  "Shrishti"  5000

        self.name = name
              ↓
    account1.name = "Shrishti"

        self.balance = balance
              ↓
    account1.balance = 5000
```

This is the key diagram for understanding the difference.

11. Example: deposit(self, amount)

```
class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

account1 = BankAccount("Shrishti", 5000)

account1.deposit(2000)

print(account1.balance)
```

```
self   → account1
amount → 2000

self.balance += amount
        ↓
account1.balance += 2000
        ↓
5000 + 2000
        ↓
7000
```

12. Same Method, Different Objects

```
class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def show_balance(self):
        print(self.name, ":", self.balance)

account1 = BankAccount("Shrishti", 5000)
account2 = BankAccount("Rahul", 10000)

account1.deposit(1000)
account2.deposit(500)

account1.show_balance()
account2.show_balance()
```

Call	self	Result
account1.deposit(1000)	account1	account1.balance becomes 6000
account2.deposit(500)	account2	account2.balance becomes 10500

13. Parameter vs Attribute

A parameter and an attribute can contain the same value, but they are not the same thing.

```
class BankAccount:

    def __init__(self, name):
        self.name = name

account1 = BankAccount("Shrishti")

print(account1.name)
```

Here name is a **parameter**, while self.name is an **attribute**. The parameter is used during the call; the attribute remains part of the object's state.

14. Parameter vs self vs Attribute — Side by Side

Concept	Example	What is it?	Main purpose
Parameter	name	Local name in method definition	Receives an input value
self	self	Reference to current object	Identifies the object being used
Attribute	self.name	Data stored on the object	Stores object state

Memory trick: Parameter = receives. self = identifies. Attribute = stores.

15. Why Can't We Just Use name Instead of self.name?

```
class BankAccount:

    def __init__(self, name):
        name = name
```

This only assigns the local parameter back to itself. It does **not** create an attribute on the object.

```
class BankAccount:

    def __init__(self, name):
        self.name = name
```

Now the value becomes part of the object's state and can later be accessed with account1.name.

16. Another Example: Student

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

    def add_bonus(self, bonus):
        self.marks += bonus

student1 = Student("Shrishti", 85)

student1.display()
student1.add_bonus(5)
student1.display()
```

```
__init__(self, name, marks)

self   → current Student object
name   → parameter
marks  → parameter

self.name
self.marks
        ↓
object attributes
```

17. self Can Technically Have Another Name

This is valid Python because the first parameter is simply a parameter name:

```
class Student:

    def __init__(current_student, name, marks):
        current_student.name = name
        current_student.marks = marks

    def display(current_student):
        print(current_student.name)
        print(current_student.marks)
```

But the standard version is:

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print(self.name)
        print(self.marks)
```

Important: self is not a reserved keyword, but changing it is strongly discouraged for readability and consistency.

18. Common Mistake: Forgetting self

```
class BankAccount:

    def __init__(self, name, balance):
        name = name
        balance = balance
```

The object does not receive these values as attributes. The correct version is:

```
class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
```

The self. prefix tells Python that the data belongs to the current object.

19. Common Mistake: Treating self as the Object's Name

Do not think of self as the text "account1". It is a reference to the actual current object.

```
account1 = BankAccount("Shrishti", 5000)

# During account1.show_balance():
# self refers to the account1 object.
```

If another object calls the same method, self refers to that other object.

20. Complete Example to Trace

```
class BankAccount:

    bank_name = "ABC Bank"

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def show_details(self):
        print("Bank:", self.bank_name)
        print("Name:", self.name)
        print("Balance:", self.balance)


account1 = BankAccount("Shrishti", 5000)
account1.deposit(2000)
account1.show_details()
```

Name	Role
BankAccount	Class
account1	Object / instance
self	Reference to account1 while its instance method runs
name	Parameter of __init__()
balance	Parameter of __init__()
amount	Parameter of deposit()
self.name	Attribute belonging to account1
self.balance	Attribute belonging to account1
self.bank_name	Access to the class attribute through the current object

21. Quick Comparison

Question	Parameter	self	Attribute
Example	name	self	self.name
What is it?	Input variable	Current-object reference	Object data
Main role	Receives input	Identifies current object	Stores object state
Where used?	Function/method definition	Instance method	Object/class state
Example after call	name → "Shrishti"	self → account1	account1.name → "Shrishti"

22. Final Mental Model

```
class BankAccount:

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount


account1 = BankAccount("Shrishti", 5000)
account1.deposit(2000)


                 METHOD CALL
                      |
                      v
            account1.deposit(2000)
                      |
        +-------------+-------------+
        |                           |
      self                       amount
        |                          |
        v                          v
    account1                    2000
        |
        +----> self.balance
        |
        v
 account1.balance

In __init__():
    name → parameter
    self.name → object attribute

In deposit():
    amount → parameter
    self.balance → object attribute
```

23. Interview / Revision Points

1. self is the conventional name for the first parameter of an instance method.
2. self can technically be renamed, but using self is the standard Python convention.
3. A parameter is a local name that receives a value supplied to a function/method.
4. An attribute is data associated with an object or class.
5. self.name is an attribute; name by itself can be a parameter.
6. self identifies the current object on which an instance method is operating.
7. self.name = name means: store the received name value in the current object's name attribute.
8. The same method works with different objects because self refers to whichever object called it.
9. self is not a reserved keyword, but changing it is strongly discouraged for readability.
10. **Memory rule:** Parameter receives → self identifies → attribute stores.

24. One-Line Summary

When a method is called, self identifies the current object, parameters receive input values, and expressions such as self.name access or store data belonging to that object.